



PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

PROGRAMMING WITH PYTHON

Exception Handling, File I/O, Modules and Packages

User-defined Exceptions

Lekha A

Computer Applications



PROGRAMMING WITH PYTHON

Custom Exceptions

- Define custom exceptions by creating a new class that is derived from the built-in Exception class.



PROGRAMMING WITH PYTHON

Custom Exceptions

- `class CustomError(Exception):`
- `...`
- `try:`
- `...`
- `except CustomError:`
- `...`



PROGRAMMING WITH PYTHON

Custom Exceptions

```
# define Python user-defined exceptions
class InvalidAgeException(Exception):
    "Raised when the input value is less than 18"
    pass

# you need to guess this number
number = 18

try:
    input_num = int(input("Enter a number: "))
    if input_num < number:
        raise InvalidAgeException
    else:
        print("Eligible to Vote")

except InvalidAgeException:
    print("Exception occurred: Invalid Age")
```

Enter a number: 12

Exception occurred: Invalid Age



PROGRAMMING WITH PYTHON

Custom Exceptions

```
class SalaryNotInRangeError(Exception):  
    """Exception raised for errors in the input salary.  
  
    Attributes:  
        salary -- input salary which caused the error  
        message -- explanation of the error  
    """  
  
    def __init__(self, salary, message="Salary is not in (5000, 15000) range"):  
        self.salary = salary  
        self.message = message  
        super().__init__(self.message)  
  
salary = int(input("Enter salary amount: "))  
if not 5000 < salary < 15000:  
    raise SalaryNotInRangeError(salary)
```



PROGRAMMING WITH PYTHON

Custom Exceptions

- Enter salary amount: 1000

```
-----  
SalaryNotInRangeError          Traceback (most recent call last)  
Cell In[46], line 17  
     15 salary = int(input("Enter salary amount: "))  
     16 if not 5000 < salary < 15000:  
--> 17     raise SalaryNotInRangeError(salary)  
  
SalaryNotInRangeError: Salary is not in (5000, 15000) range
```



PROGRAMMING WITH PYTHON

Raise Exceptions

- Python also provides the raise keyword to be used in the context of exception handling.
- It causes an exception to be generated explicitly.
- Built-in errors are raised implicitly.
- However, a built-in or custom exception can be forced during execution.



PROGRAMMING WITH PYTHON

Raise Exceptions

- The raise statement in Python is used to raise an exception.
- Try-except blocks can be used to manage exceptions, which are errors that happen while a programme is running.
- When an exception is triggered, the program goes to the closest exception handler, interrupting the regular flow of execution.



PROGRAMMING WITH PYTHON

Raise Exceptions

- The raise keyword is typically used inside a function or method, and is used to indicate an error condition.
- An exception can be thrown and immediately halt the running of the program by using the raise keyword.
- Python looks for the closest exception handler, which is often defined using a try-except block, when an exception is triggered.



PROGRAMMING WITH PYTHON

Raise Exceptions

- If an exception handler is discovered, its code is performed, and the try-except block's starting point is reached again.
- If an exception handler cannot be located, the software crashes and an error message appears.



PROGRAMMING WITH PYTHON

Raise Exceptions

```
try:
    x,y=100,2
    z=x/2
    if z > 10:
        raise ValueError(z)
except ValueError:
    print(z, "is out of allowed range")
else:
    print(z, "is within the allowed range")
```

50.0 is out of allowed range



PROGRAMMING WITH PYTHON

Raise Exceptions

```
def divide(a, b):  
    if b == 0:  
        raise Exception(" Cannot divide by zero. ")  
    return a / b  
  
try:  
    result = divide(10, 0)  
except Exception as e:  
    print(" An error occurred: ", e)
```

An error occurred: Cannot divide by zero.



PROGRAMMING WITH PYTHON

Raise Exceptions

- ```
def check_age(age):
 if age < 18:
 raise Exception(" Age must be 18 or above. ")
 else:
 print(" Age is valid. ")
```

```
try:
 check_age(17)
except Exception as e:
 print(" An error occurred: ", e)
```

An error occurred: Age must be 18 or above.



# PROGRAMMING WITH PYTHON

## Raise Exceptions

```
x = "hello"
```

```
if not type(x) is int:
 raise TypeError("Only integers are allowed")
```

```

TypeError Traceback (most recent call last)
Cell In[55], line 4
 1 x = "hello"
 3 if not type(x) is int:
----> 4 raise TypeError("Only integers are allowed")

TypeError: Only integers are allowed
```



# PROGRAMMING WITH PYTHON

## Raise Exceptions

```
x = -1

if x < 0:
 raise Exception("Sorry, no numbers below zero")
```

```

Exception Traceback (most recent call last)
Cell In[56], line 4
 1 x = -1
 3 if x < 0:
----> 4 raise Exception("Sorry, no numbers below zero")

Exception: Sorry, no numbers below zero
```





# PROGRAMMING WITH PYTHON

## Advantages of Exceptions

- Improved program reliability
  - By handling exceptions properly, prevent the program from crashing or producing incorrect results due to unexpected errors or input.
- Simplified error handling
  - Exception handling allows the separation of error handling code from the main program logic, making it easier to read and maintain the code.



# PROGRAMMING WITH PYTHON

## Advantages of Exceptions

---

- Cleaner code
  - With exception handling, avoid using complex conditional statements to check for errors, leading to cleaner and more readable code.



# PROGRAMMING WITH PYTHON

## Advantages of Exceptions

---

- Easier debugging
  - When an exception is raised, the Python interpreter prints a traceback that shows the exact location where the exception occurred, making it easier to debug the code.



# PROGRAMMING WITH PYTHON

## Disadvantages of Exceptions

- Performance overhead
  - Exception handling can be slower than using conditional statements to check for errors, as the interpreter has to perform additional work to catch and handle the exception.



# PROGRAMMING WITH PYTHON

## Disadvantages of Exceptions

- Increased code complexity
  - Exception handling can make the code more complex, especially if there is a need to handle multiple types of exceptions or implement complex error handling logic.



# PROGRAMMING WITH PYTHON

## Disadvantages of Exceptions

- Possible security risks
  - Improperly handled exceptions can potentially reveal sensitive information or create security vulnerabilities in the code, so it's important to handle exceptions carefully and avoid exposing too much information about the program.



# PROGRAMMING WITH PYTHON

## Recap

---

- Custom Exceptions
- Raise Exceptions
- Advantages of Exceptions
- Disadvantages of Exceptions



**THANK YOU**

---

**Lekha A**

Computer Applications